

Migration strategies from monolithic systems to Kubernetes-native microservices in enterprise environments

Shubham Srivastava *

Integral University, Lucknow, Uttar Pradesh, India.

World Journal of Advanced Engineering Technology and Sciences, 2026, 20(01), 016-025

Publication history: Received on 01 May 2026; revised on 30 June 2026; accepted on 02 July 2026

Article DOI: <https://doi.org/10.30574/wjaets.2026.20.1.0329>

Abstract

As legacy application architectures can limit the ability to deliver quickly, engineer for resilience, scale, and deploy independently, enterprise migration to microservices running on Kubernetes has emerged as a key modernization priority. This review analyzes peer-reviewed research in the field of migration patterns, service decomposition, containerization, orchestration, technical debt, organizational readiness and runtime governance in enterprise contexts. The literature suggests that full architectural replacement is seldom the most effective migration path. Instead, incremental extraction, a well-defined service-boundary understanding, platform engineering capability, observability maturity, data ownership redesign, and a staged coexistence between legacy and cloud-native elements are essential for durable modernization. Reported studies also highlight current shortcomings, such as poor longitudinal evidence, inconsistent migration measures, only minimal empirical validation for Kubernetes, and inadequate attention to regulated enterprise constraints. The review builds an abbreviated conceptual framework for migration to Kubernetes and draws inferences from comparative results from reported studies. It concludes that there needs to be more evidence in the field on production-scale economics of migration, platform reliability, socio-technical governance, and measurable results in different and diverse enterprise domains.

Keywords: Cloud-Native Migration; Kubernetes; Legacy Modernization; Microservices; Monolithic Systems; Service Decomposition

1. Introduction

In response to the growing demands of enterprise applications to scale elastically, tolerate faults, and operate across cloud and hybrid platforms, migration from monolithic applications to Kubernetes-native microservices has become a critical research and industrial challenge. The issue of migration is not only about the technology being used for deployment. It shifts architectural responsibility from a single application runtime to a distributed topology in which service boundaries, container lifecycle management, traffic routing, observability, configuration, and policy enforcement become central architectural concerns [1]. Later on, a better understanding of the microservices was provided, where some issues were not yet resolved, such as the granularity of services, operational complexity, testing, data consistency, and distributed failure modes [2].

Kubernetes extends this migration agenda by codifying cloud-native principles through the cloud-native tenets into pods, deployments, services, ingress control, health probes, secrets, configuration objects, horizontal scaling, and declarative reconciliation. The literature suggests that, after a decade of cloud computing, portability, automated management, and runtime elasticity emerged as organizing principles for cloud-native applications [3]. Similarly, containerization scholarship shows that containers support packaging, deployment repeatability, and resource isolation, while also introducing scheduling, networking, storage, and security concerns [4]. Kubernetes can then be

* Corresponding author: Shubham Srivastava.

seen not just as an end-to-end platform but also as an architectural catalyst that reveals the hidden coupling, unclear ownership, and lack of automation in monolithic enterprise systems.

The academic interest of this subject is linked to software architecture, empirical software engineering, cloud computing, information systems modernization and organizational studies. Based on 196 papers, systematic mapping of microservice architecture research reveals a growing body of knowledge for microservice architecture design principles, patterns, quality attributes and deployment environments but also identifies a lack of balance between conceptual guidance and rigorously measured industrial outcomes [5]. This imbalance is relevant for enterprise migration as modernization initiatives must justify architectural risk by demonstrating gains in deployability, scalability, maintainability, compliance, and reliability as measured by these gains. A successful strategy in a small digital service can turn into a failure in a regulated enterprise portfolio, as long as shared databases, batch workloads, mainframe dependencies, audit controls and central release boards are still in place.

A major gap addressed in this review is the lack of integration of monolith decomposition literature with the Kubernetes-native operational literature. Some studies discuss microservice migration as refactoring and others discuss the runtime infrastructure as containers. Very few peer-reviewed works provide a true journey from monolithic enterprise systems, to extraction of services, then to containerization, to Kubernetes deployment, to progressive delivery, to production governance. There is also a lack of consistency of evidence reported in experience reports, practitioner surveys, mapping studies and performance evaluations, which makes comparison hard. This means that enterprise decision makers get substantial architectural discussion but limited empirical data to choose from in deciding between competing migration choices, including hybrid coexistence.

The aim of this review is to critically analyze peer-reviewed publications from 2015 and later that directly contribute to understanding migration strategies from monolithic systems to microservices deployed on Kubernetes in enterprise environments. The review highlights reported migration practices, methodological approaches, comparative results, operational considerations and gaps in research. Its contribution is a concise scholarly discussion that links service decomposition with the adoption of Kubernetes-native platforms, without treating microservices as a generic modernization label.

2. Literature Review

There are four overlapping clusters of literature: migration patterns, empirical motivations and barriers, foundations of cloud-native containers, and operational quality concerns. There is clear guidance to move from monolithic systems to microservices from migration-pattern research. The general approach of pattern-oriented work is to look at incremental extraction and coexisting legacy and new services, adapter-based integration, and database refactoring as workable alternatives to large-scale rewrites of the systems in question [6]. The banking domain evidence confirms that regulatory requirements, legacy integration, organization inertia, and requirements for enterprise stability when providing stable customer-facing services during architectural transitions are constraints on enterprise migration. Other factors that are mentioned in empirical research on motivation for migration are scalability, maintainability, deployability and team autonomy, as well as service identification, data management, testing and monitoring challenges [8]. The most defensible interpretation is that microservices do not represent the endpoint of architectural change; rather, migration requires a sequenced architectural transition that preserves delivery continuity.

The second cluster is on costs, benefits and socio-technical barriers. Research on microservices pros and cons shows that the advantages of independent deployment and scalability in microservice architectures have drawbacks like the burden of coordination overhead, distributed debugging, handling service dependencies and complexity of infrastructure [9]. The results of professional research also indicate that technical and organizational issues like employee skill, deployment automation and organizational readiness impact microservices adoption [10]. Migration from monolith to microservices is further complicated by technical-debt studies, which reveal that moving from monolith to microservices does not necessarily decrease the amount of technical debt but it can migrate from code structure to service interfaces, infrastructure setups, and operational scripts [11]. It is an important cluster to consider when transitioning to Kubernetes since orchestration can be helpful in repeatability, but can also contribute to configuration debt, security surface area, and runtime diagnostic requirements.

A third cluster concerns decomposition as well as modernization techniques. Service-identification research classifies decomposition approaches according to business capabilities, static code analysis, dynamic traces, semantic analysis, data dependencies, and hybrid assessments. A number of initial studies have shown that it is helpful to systematically map monolith to microservices, either by code or domain modeling, and the findings can only be validated after deployment of the resulting services, on a relatively small scale [13]. They are crucial to the Kubernetes-native migration

because rigid boundaries lead to a lot of “chatty” communication, wasteful resource utilization, inflexible scaling policies, and dependency between services in the cluster. Kubernetes can reschedule containers, scale replicas, but does not help with bad domain partition, nor unknown transactional dependency.

The fourth cluster is on cloud-native and container-operation foundations. Studies on container orchestration identify scheduling, service discovery, health checks, elasticity, and rollout control as critical features of modern platforms [14]. DevOps-oriented container work coupled with automation and Infrastructure governance, is observed to be enabled by containerized deployment with repeatable environment and quick delivery pipelines. For cloud-native applications managed by themselves, the situation is compounded by the need for adaptation loops, monitoring and auto-recovery [16] properties of the applications, to sustain their operation. The studies that target the practices of the practitioners, such as observability, contract testing, and deployment verification, validate that observability, contract testing, and deployment verification become essential as microservice estates scale [17]. Comparative performance analysis also shows that microservices can enhance the scalability given the appropriate workloads but can also add overhead in communication when compared to monolithic deployment [18]. Taken together, the reviewed evidence suggests that Kubernetes-native migration should be understood as both a redesign and an operational transformation, rather than merely as refactoring.

Table 1 Summary of key findings

Ref	Focus	Key Findings
[6]	Migration patterns	Incremental extraction and coexistence patterns reduce modernization risk compared with replacement-oriented migration.
[7]	Banking-domain migration	Enterprise migration requires regulatory continuity, integration stability, and staged architectural change.
[8]	Migration motivations and issues	Scalability and deployability motivate migration, while service identification and monitoring remain persistent obstacles.
[9]	Microservice pains and gains	Benefits in autonomy and scalability are counterbalanced by distributed complexity and operational overhead.
[10]	Adoption drivers and barriers	Staff capability, automation maturity, and organizational readiness strongly condition adoption outcomes.
[11]	Technical debt after migration	Migration can relocate debt into interfaces, infrastructure configuration, and test environments.
[12]	Service identification	Hybrid decomposition methods better capture code, data, and business capability dependencies.
[13]	Refactoring approaches	Migration research emphasizes extraction methods but offers limited longitudinal production validation.
[14]	Container orchestration	Orchestration platforms enable scheduling, recovery, service discovery, and elastic deployment control.
[15]	Containerized DevOps	Container pipelines strengthen deployment repeatability but require disciplined infrastructure automation.
[16]	Self-managing cloud-native systems	Monitoring-driven adaptation and recovery mechanisms are central to cloud-native operation.
[17]	Design, monitoring, testing	Production microservices require observability, contract checks, and automated verification.
[18]	Performance and scalability	Microservices may scale better under suitable load but can incur network and coordination overhead.

Table 1 indicates that there is no uniform migration strategy. Contingent migration design, where architectural decomposition, platform capability and enterprise governance unfold collaboratively, is more likely to be the right approach. One recurrent challenge found in all of the studies is the scarcity of longitudinal evidence showing the relation between migration decisions and enduring effects in the production systems that are managed by Kubernetes.

3. Methodology

The reviewed literature includes qualitative experience reports, systematic mappings, professional surveys, taxonomies, and controlled performance evaluations. Experience reports add high contextual validity as enterprise migration is heavily influenced by legacy constraints, compliance processes, team boundaries and release governance. For this reason, the literature on banking migration is relevant, as it demonstrates the process of modernization in the face of constraints on operational continuity and not in a greenfield environment [7]. Its limitation is the narrow scope of generalizability; the ability of one domain to provide evidence for whether or not observed practices transfer to systems in manufacturing, retail, healthcare, or public sector.

Systematic mapping and review studies offer general overview of migration trends, architectural issues, and gaps [9, 13]. They are useful for classification, terminology stabilization, and theme identification. They have the drawback that they rely on primary studies that are of diverse quality and have different measurements, making causal inference difficult. The method of identification of services from a taxonomy of services is introduced by comparing the code-centric approach, data-centric approach, domain-centric approach with the hybrid approach for decomposing the service [12]. Many approaches in this vein, however, halt before deploying Kubernetes, and assume that the services identified are associated with desirable run-time properties including efficient autoscaling, acceptable latency, and manageable observability cost.

Practitioner motives, barriers, and quality issues are reported in survey-based studies [8, 10, 17]. These studies can be helpful to reveal factors that are socio-technical and that are not captured in purely architectural models. The drawback of their limitation is relying on self-reported experience which can lead to an over-estimate or under-estimate of the perceived benefit or hidden cost. Rather, controlled performance comparisons provide empirical contrast between monolithic and microservice architectures [18]. This type of effort helps explain the compromises and trade-offs of scalability and overhead, but as the systems used for experiments get more complex, they often miss enterprise considerations like shared databases, service mesh overhead, legacy message brokers, identity systems, audit logging, and multi-team deployment pipelines.

Table 2 Method comparison

Ref	Method	Strengths	Limitations
[6]	Pattern extraction from migration experience	Captures practical sequencing of monolith decomposition and coexistence.	Limited quantitative validation of each pattern under Kubernetes runtime constraints.
[7]	Industrial experience analysis	Shows migration under regulatory and enterprise continuity conditions.	Domain specificity restricts transferability.
[8]	Empirical investigation with practitioners	Identifies real motivations and recurring migration obstacles.	Self-reporting limits outcome verification.
[11]	Technical-debt assessment	Challenges assumptions that migration inherently reduces maintainability debt.	Debt measures may miss Kubernetes configuration and platform debt.
[12]	Taxonomic classification	Clarifies decomposition inputs such as code, data, and business capabilities.	Runtime validation after orchestration remains limited.
[13]	Systematic mapping	Organizes refactoring approaches and research maturity.	Primary-study heterogeneity weakens causal comparison.
[14]	Container orchestration analysis	Defines platform capabilities relevant to Kubernetes-native migration.	Less attention to legacy enterprise decomposition.
[17]	Practitioner-centered quality study	Connects design, monitoring, and testing with production operation.	Limited direct measurement of platform-level outcomes.
[18]	Comparative performance evaluation	Provides concrete contrast between monolithic and microservice behavior.	Experimental workloads simplify enterprise dependency complexity.

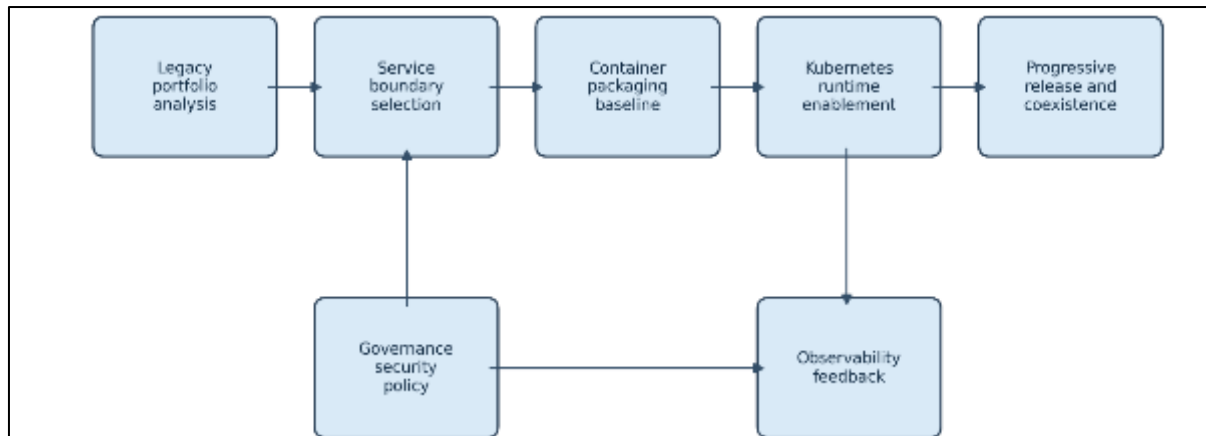


Figure 1 Kubernetes-native migration decision framework

Kubernetes-native migration is not a linear rewrite, but rather a gated modernization path, as illustrated in Figure 1. It is relevant because the literature has demonstrated repeatedly that a decomposition and platform enablement process will not result in a distributed monolith unless there is coordination with runtime feedback. The framework connects legacy analysis with service-boundary selection, Kubernetes platform enablement, progressive release and runtime learning with governance gates. The interpretative model is derived from the literature on migration-pattern, decomposition, orchestration and operations [6, 8, 12, 14, 16, 17].

4. Discussion

The findings reported indicate that a staged approach to architectural risk reduction, as a migration strategy from monolithic systems to Kubernetes-native microservices, is the most strongly supported path in the reviewed literature. Studies based on pattern and experience agree that incremental migration is important particularly when there are stable revenue-generating functions in monolithic systems [6, 7]. Selected capabilities move toward independently deployable services, while the monolith continues to run, with the help of the strangler style, branch-by-abstraction, façade isolation and anti-corruption layers. This is the approach that is compatible with Kubernetes, as extracted services can be packaged, deployed, scaled, monitored, and rolled back without needing to convert the entire portfolio onto Kubernetes. These benefits, however, come with disciplined integration design. However, if the services that are extracted are still dependent on common database schemas, synchronous calls to monolithic internals, or common release windows, Kubernetes offers deployment mobility but not true architectural independence.

The other significant trend is aspects of service decomposition that do not quite fit into the production platform. There are many different approaches that have been proposed to service-identification taxonomies, such as business-capability mapping, code dependency analysis, semantic clustering, dynamic trace analysis, and data-flow assessment [12]. None of these methods is enough in Enterprise migration. Preserving business meaning is possible with domain-based decomposition, though it can be difficult to capture technical coupling that may be embedded in legacy code. Dependencies can be identified by static analysis, which may not capture runtime behavior. Data-oriented approaches show transactional coupling, but may not show well enough the capability boundaries of customer-facing transactions. Another layer of evaluation comes from the Kubernetes-native targets: service boundaries should be auto-scalable, failure-contained, health-check semantic, deployment independent, and have a clear ownership of the configuration. There are no validated metrics in the literature that correlate a proposed boundary to the quality of the Kubernetes runtime.

Findings on technical debt challenge overly optimistic migration narratives. The evidence that migration might not eliminate technical debt implies that debt often manifests itself in different forms, rather than being fully eradicated [11]. Debt can manifest itself in a monolith as modules that are intertwined, code duplication, or slow releases. Once migrated, similar pressure can be manifested as Helm chart sprawl, inconsistent deployment templates, too many calls across services, uncontrolled configuration, brittle observability, and incompatible API versions. While Kubernetes can make infrastructure declarative, declarative infrastructure becomes software that needs to be versioned, reviewed, tested, secured, and owned. Therefore, a migration strategy geared towards service count or container adoption can only achieve success at the expense of architectural fragmentation instead of maintainability.

Empirical research and surveys have shown that organizational readiness significantly influences migration outcomes [8, 10, 17]. Microservices spread out the responsibility of the architecture, and Kubernetes spreads out the responsibility of the operations, through platform abstractions. A centralized model without platform engineering can lead to challenges in cluster provisioning, namespace management, ingress policy, secret management, and incident response. On the other hand, deploying without any precautions can result in inconsistent practices and security risks. The most credible model in the literature is the hybrid model: the platform team owns paved roads, reusable templates, observability standards, and policy automation, while the Service team owns Code and Deployment Intent and Service-level Behavior. This is a model tailored for Kubernetes-native migration as well as socio-technical architecture, not only as an infrastructure migration.

Evidence of performance and scalability is complex. A comparative study reveals that microservices can enable horizontal scaling and localised resource allocation, but if they are too fine-grained and too synchronous, the overhead of communication can decrease efficiency [18]. While Kubernetes makes it easier to manage replicas and recover from failures, it does not remove serialization cost, network latency, connection management or complexity of distributed transactions. It may be assumed that the workload shape needs to be considered before going into the decomposition phase in migration strategies. Kubernetes autoscaling and rolling deployment can be useful for stateless request-processing services that have fluctuating workloads. Conversely, for tightly coupled transactional workloads, service extraction might not be possible until the services are modularized in monoliths, the database is redesigned, asynchronous messaging is introduced, or refactoring is performed by using events.

Studies of container and orchestration processes explain the migration substrate. Readiness probes, liveness probes, resource requests and limits, autoscaling policies, separation of configuration, secrets management, immutability of images, and control of rollout strategies are all necessary for Kubernetes-native modernization [14], [15]. The mechanisms are not an after-thought, they are part of the service's operational contract that is being migrated. Also, a study into self-managing cloud-native applications has shown that runtime feedback loops and capability for automated recovery are key to maintaining reliability when systems become more complex [16]. This discovery doesn't just involve the code extraction tasks – the migration backlog needs to cover the tasks of telemetry, alerting, chaos testing, deployment verification, and incident playbooks, as well.

The results of the studies reviewed are compared in Table 3. Migration value is multi-dimensional and covers the areas of maintainability, scalability, delivery cadence, reliability and governance. There is no study that definitively shows that one strategy is superior in every enterprise environment.

Table 3 Results comparison

Ref	System / Context	Metric / Basis of Comparison	Outcome
[6]	Cloud-native migration cases	Pattern applicability and migration sequencing	Incremental extraction reduced replacement risk and supported staged service rollout.
[7]	Banking enterprise system	Continuity, integration, and compliance constraints	Migration required coexistence patterns and domain-sensitive rollout control.
[8]	Practitioner migration experience	Motivations, issues, and migration process	Scalability and maintainability gains were constrained by monitoring and decomposition difficulty.
[11]	Migrated monolithic system	Technical-debt comparison	Debt did not automatically decline; debt categories shifted after migration.
[12]	Legacy modernization approaches	Service identification basis	Hybrid methods offered broader boundary evidence than single-source analysis.
[14]	Container orchestration platforms	Scheduling, scaling, recovery, service management	Orchestration enabled runtime control but required platform-level expertise.
[17]	Microservice practice	Design, monitoring, and testing concerns	Observability and verification practices were central to reliable operation.
[18]	Monolith and microservice comparison	Performance and scalability behavior	Microservices improved selected scalability outcomes but introduced communication overhead.

The source pool evidence is summarized by the major topic concerns in Figure 2. It emphasizes that migration patterns and architecture studies are more common than actual production evaluations of Kubernetes, which is why there is not much certainty of what happens when you migrate to different platforms. The lengths of the bars indicate the number of reviewed peer-reviewed articles associated with each primary concern. The classification represents the authors' manual categorization of the dominant contribution of references [1]–[18].

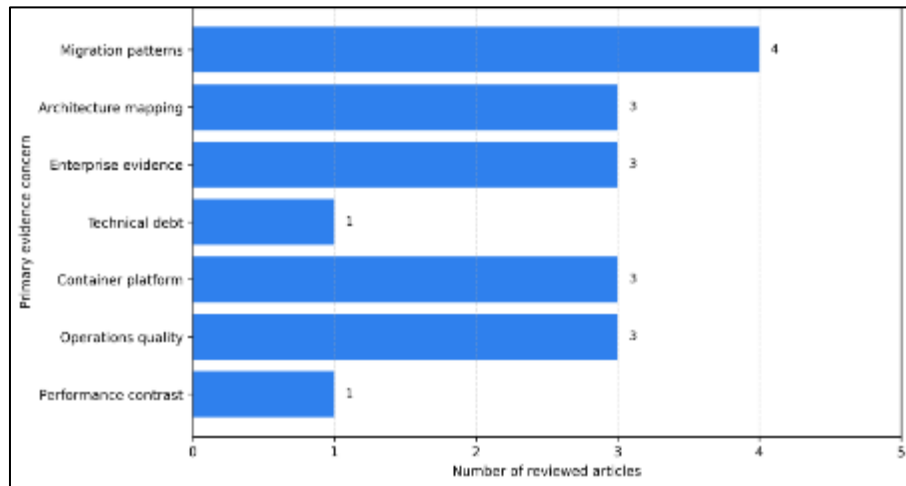


Figure 2 Evidence distribution across migration concerns

Four migration strategies are compared in Figure 3 to the criteria developed based on the common findings in the studies reviewed. The figure is important because enterprise migration strategy is a trade-off surface: less risky strategies might yield slower architectural separation; more aggressive extraction strategies means more coordination and platform requirements. Scores range from 1 to 5, with higher scores indicating stronger support for the criterion. The values are interpretative ratings derived from migration reports, adoption barriers, orchestration constraints, and performance findings. Comparative analysis of references [6]–[18].

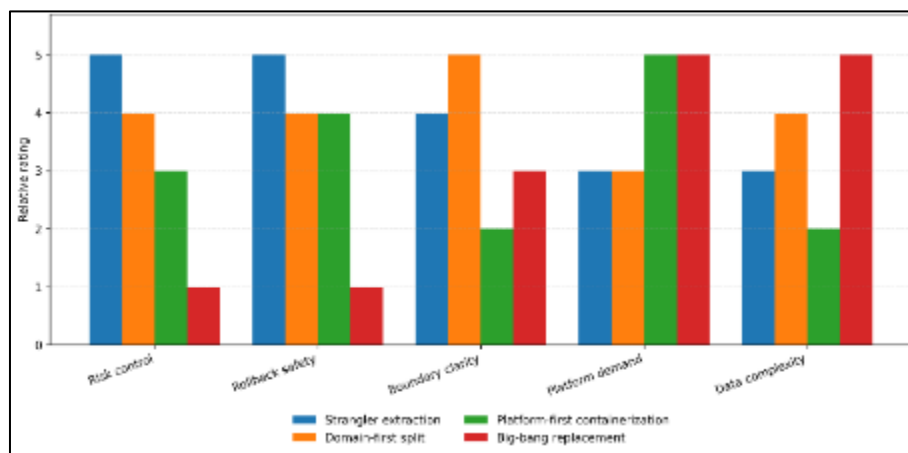


Figure 3 Comparative strategy profile for enterprise migration

Technical coupling and market change rate are combined into a migration sequencing decision matrix as shown in Figure 4. It's not a one-size-fits-all approach and demonstrates why enterprise considerations drive the need to move in different ways when migrating to Kubernetes. The matrix details architectural coupling and market change rate to four enterprise migration starting points. Decomposition, migration-pattern, container-orchestration, and operations findings are integrated from the reviewed literature [6]–[18].

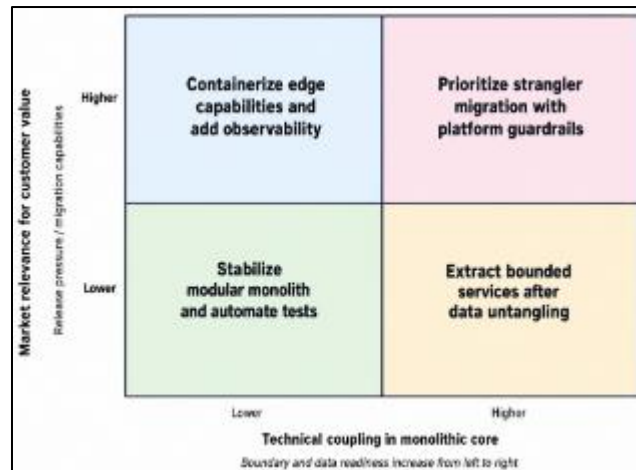


Figure 4 Migration sequencing matrix for Kubernetes-native modernization

The implications of the analysis are three-fold. Architectural independence should be the first criterion for assessing Kubernetes-native migration, not the number of containers. Second, the governance of migration needs to monitor operational readiness, such as service-level goals, traceability, deployment safety and owning manifests and policies. Thirdly, research must differentiate among partial containerization of a monolith, service extraction without independent data ownership, and full deployment of autonomous Kubernetes-native microservices. If these states are treated as similar, the real journey towards enterprise modernization is hidden.

5. Future Directions

Future research should conduct longitudinal studies of enterprise migration programs from monolithic systems through service extraction, Kubernetes deployment, and post-migration operation. There are useful mappings and experience reports to be found in the current literature, but there is limited evidence linking migration decisions to subsequent outcomes (e.g., incident rate, mean time to recovery, deployment frequency, cost efficiency, security posture, maintainability after several release cycles) [11, 13, 17]. This research needs to delineate the temporary cost of transition from permanent building cost.

The next priority is to develop metrics for decomposing into Kubernetes. There are service identification mechanisms provided in the existing solutions that classify inputs and boundary logic; however, more effective validation against runtime properties, such as autoscaling efficiency, inter-service latency, service mesh overhead, pod restart behavior, namespace isolation, and observability cost is required [12], [14]. Recommendations for the decomposition of code should not only consider code modularity but also fit into the Kubernetes cluster in use. In this direction, static modelling of the platform could be linked to measurable platform behaviour.

A third direction relates to the governance of migration in regulated enterprises. Organizational barriers and technical-debt relocation are acknowledged in literature, but more precise models are required in the context of auditability, policy-as-code, identity federation, secret rotation, software supply-chain control and compliance evidence in Kubernetes-native environments [10] [15] [17]. The responsibility of platform teams, security teams and application teams should be explored in relation to the avoidance of the creation of a central release bottleneck.

A fourth is through a comparative assessment of migration strategies. Common metrics, not case narratives, should be used to compare Strangler migration, domain-first decomposition, platform-first containerization, and hybrid coexistence. Existing research reveals that microservices can enhance scalability at the expense of communication overhead [18]. Similar comparative designs should test reliability, cost, maintainability, and delivery cadence using realistic enterprise workloads.

Finally, there are cross-disciplinary efforts between software architecture, cloud operations, organizational design, and information systems governance. The migration to Kubernetes is a socio-technical system where architecture diagrams, cluster policies, team ownership, deployment pipelines and runtime telemetry influence each other. Future research work which combines these layers will give better guidance than code extraction or orchestration mechanics.

6. Conclusion

There is a need for coordinated architectural, operational and organizational change when moving from monolithic to Kubernetes-native microservices. Based on the literature reviewed, the following are identified as key elements to successful enterprise modernization: incrementally introducing new services and service boundaries, careful selection of the service boundaries, delivery automation of services using containers, observability, and platform governance. Kubernetes provides substantial capabilities for scheduling, rollout control, recovery, and scaling, but it cannot fix a poor decomposition, shared-data coupling, weak testing, or unclear ownership.

A central conclusion is that the value of migration does not arise merely from the presence of microservices and containers, but because the transition path itself is mature. Organizations can build distributed versions of current monolithic issues if they break down systems without taking into account data ownership, deployment safety, and observability. On the other hand, businesses can enhance delivery independence, operational resilience, and mitigate migration risks by implementing staged extraction alongside platform engineering and governance.

Several research gaps remain. There is a lack of longitudinal evidence in the field, Kubernetes-specific migration metrics, and fair comparative evaluations in regulated enterprise settings. Future scholarships should examine the links between service identification and runtime behavior, technical debt and platform configuration, and migration strategy and measurable production outcomes. The most viable path is a disciplined and evidence-based migration approach in which monolithic modernization, Kubernetes-native operation, and enterprise governance are all a single transformation.

References

- [1] Balalaie, A., Heydarnoori, A., & Jamshidi, P. (2016). Microservices architecture enables DevOps: Migration to a cloud-native architecture. *IEEE Software*, 33(3), 42–52.
- [2] Jamshidi, P., Pahl, C., Mendonça, N. C., Lewis, J., & Tilkov, S. (2018). Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3), 24–35.
- [3] Kratzke, N., & Quint, P.-C. (2017). Understanding cloud-native applications after 10 years of cloud computing: A systematic mapping study. *Journal of Systems and Software*, 126, 1–16.
- [4] Pahl, C., Brogi, A., Soldani, J., & Jamshidi, P. (2019). Cloud container technologies: A state-of-the-art review. *IEEE Transactions on Cloud Computing*, 7(3), 677–692.
- [5] Di Francesco, P., Lago, P., & Malavolta, I. (2019). Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*, 150, 77–97.
- [6] Balalaie, A., Heydarnoori, A., & Jamshidi, P. (2018). Microservices migration patterns. *Software: Practice and Experience*, 48(11), 2019–2042.
- [7] Bucchiarone, A., Dragoni, N., Dustdar, S., Larsen, S. T., & Mazzara, M. (2018). From monolithic to microservices: An experience report from the banking domain. *IEEE Software*, 35(3), 50–55.
- [8] Taibi, D., Lenarduzzi, V., & Pahl, C. (2017). Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation. *IEEE Cloud Computing*, 4(5), 22–32.
- [9] Soldani, J., Tamburri, D. A., & Van Den Heuvel, W.-J. (2018). The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 146, 215–232.
- [10] Knoche, H., & Hasselbring, W. (2019). Drivers and barriers for microservice adoption: A survey among professionals in Germany. *Enterprise Modelling and Information Systems Architectures*, 14(1), 1–35.
- [11] Lenarduzzi, V., Lomio, F., Saarimäki, N., & Taibi, D. (2020). Does migrating a monolithic system to microservices decrease the technical debt? *Journal of Systems and Software*, 169, 110710.
- [12] Abdellatif, M., Shatnawi, A., Mili, H., Moha, N., El Boussaidi, G., Hecht, G., Privat, J., & Guéhéneuc, Y.-G. (2021). A taxonomy of service identification approaches for legacy software systems modernization. *Journal of Systems and Software*, 173, 110868.
- [13] Fritzsche, J., Bogner, J., Zimmermann, A., & Wagner, S. (2021). From monolithic systems to microservices: A systematic mapping study. *Information and Software Technology*, 137, 106600.

- [14] Khan, A. A. (2017). Key characteristics of a container orchestration platform to enable a modern application. *IEEE Cloud Computing*, 4(5), 42–48.
- [15] Kang, H., Le, M., & Tao, S. (2016). Container and microservice driven design for cloud infrastructure DevOps. *IEEE Cloud Computing*, 3(5), 52–59.
- [16] Toffetti, G., Brunner, S., Blöchliger, M., Spillner, J., & Bohnert, T. M. (2017). Self-managing cloud-native applications: Design, implementation, and experience. *Future Generation Computer Systems*, 72, 165–179.
- [17] Waseem, M., Liang, P., Shahin, M., Di Salle, A., & Márquez, G. (2021). Design, monitoring, and testing of microservices systems: The practitioners' perspective. *Journal of Systems and Software*, 182, 111061.
- [18] Blinowski, G., Ojdowska, A., & Przybyłek, A. (2022). Monolithic vs. microservice architecture: A performance and scalability evaluation. *IEEE Access*, 10, 20357–20374.